SMART ENERGY METER

with Protection, Power Factor Monitoring and Event Logging

Simulation-Based Technical Project Documentation

Prepared by

Sohan H.R

Developed and Tested in Wokwi | October 2026

Project status: Final functional validation completed in Wokwi. This report documents the tested firmware behaviour and its engineering limitations.

Contents

Contents 2
Executive Summary 4
1. Project Overview 4
1.1 Project objective 4
1.2 Problem definition 5
1.3 Scope 5
1.4 Out of scope 5
2. System Requirements and Design Basis 5
2.1 Functional requirements 5
2.2 Design constants 6
2.3 Design philosophy 6
3. Hardware and System Architecture 6
3.1 Components 6
3.2 Simulated sensing method 7
3.3 Text block diagram 7
3.4 Pin configuration 8
3.5 Software stack 8
4. Firmware Design and Working Principle 8
4.1 Main execution sequence 8
4.2 State and display priority 9
4.3 Input adjustment mode 9
4.4 Low-PF warning logic 9
4.5 Fault-latch logic 9
5. Electrical Calculations and Measured Parameters 9
5.1 Input scaling 9
5.2 Active power 9
5.3 Energy accumulation 10
5.4 Cost 10
5.5 Operating time and average power 10
5.6 Peak and maximum-current records 10
5.7 Load classification 10
5.8 Maximum demand 10
5.9 Worked example 11
6. Protection, Monitoring and Data Retention 11
6.1 Protection thresholds 11

6.2 Protection outputs 11
6.3 Dynamic fault display 11
6.4 EEPROM data storage 12
6.5 LCD information structure 12
7. Testing and Validation 13
7.1 Test methodology 13
7.2 Final 24-case validation matrix 13
7.3 Reference calculation checks 14
7.4 Result discussion 14
8. Limitations and Design Review 14
8.1 Simulation limitations 14
8.2 Known design issues before physical deployment 14
9. Future Development 15
10. Conclusion 15
References 16
Appendix A. Project Resources and Reproducibility 16
Appendix B. Engineering Notes for a Physical Prototype 16

Executive Summary

This project presents a simulation-based smart energy meter prototype developed around an Arduino Uno in the Wokwi online simulator. The system accepts three simulated electrical inputs representing voltage, load current and power factor, and combines them to calculate active power, energy consumption and an estimated electricity cost. In addition to basic metering, the firmware tracks operating time, average power, peak power, maximum current, load class and maximum demand.

The prototype also implements protection and monitoring functions normally associated with a more advanced meter or load-management device. Current above 10 A, voltage above 250 V or voltage below 200 V causes a latched protection trip. The relay is switched off and a red LED and buzzer indicate the fault. Reset is permitted only after the measured values return to the defined safe range. Low power factor below 0.70 is handled separately as a warning with 0.05 hysteresis, so the load remains connected while the red LED blinks.

Cumulative energy and selected peak, operating-time, demand and fault-event values are stored in the Arduino EEPROM. A 16×2 I2C LCD provides rotating monitoring screens, while the serial monitor provides detailed diagnostics and event messages. The final firmware was validated against a 24-case functional test plan covering normal operation, input adjustment, protection trips, recovery, safe reset, low-PF warning, calculations, demand, persistence, logging and fault latching.

The project is deliberately described as a prototype rather than a certified meter. Potentiometers are used as ideal sensor substitutes, the simulation does not interface with AC mains, and no claim is made about measurement accuracy or standards compliance. The documentation also records three design-review issues that should be addressed before a physical deployment: frequent EEPROM writes, long-term floating-point energy accumulation, and the narrow reachability of the HEAVY LOAD classification band.

Resume-ready positioning: This project demonstrates embedded C/C++ programming, analog input scaling, electrical power calculations, protection logic, state latching, EEPROM data retention, HMI design, event logging and systematic functional testing in a safe simulation environment.

1. Project Overview

1.1 Project objective

The objective is to design and validate a compact smart energy-metering controller that combines measurement, calculation, protection, indication, display and retained logging in one embedded firmware application. The emphasis is not on a commercial billing meter, but on demonstrating how these functions interact and how the resulting control logic can be verified safely before a physical hardware implementation.

1.2 Problem definition

A basic energy meter provides cumulative energy information, but a useful embedded monitoring device can also show live electrical quantities, identify abnormal supply or load conditions, disconnect a load when required, maintain event counts and preserve important statistics. Implementing all of these functions together introduces state-management and timing problems that are difficult and unsafe to explore directly on mains voltage. A simulator therefore provides an appropriate environment for first-stage development.

1.3 Scope

- Single-phase metering logic for one simulated load.
- Voltage range: 180–260 V, current range: 0–15 A, power-factor range: 0.50–1.00.
- Overcurrent, overvoltage and undervoltage protection with latched fault behaviour.
- Low power-factor warning with hysteresis and no load disconnection.
- Energy, cost, operating-time, average-power, peak, maximum-current and maximum-demand tracking.
- EEPROM retention of cumulative values and fault-event counters.
- Wokwi-only functional validation; no mains connection or physical-load testing.

1.4 Out of scope

- True RMS sensing, waveform sampling, phase-angle measurement and measurement calibration.
- Certified energy metering accuracy or compliance testing.
- Wireless communication, cloud dashboards and real-time clock timestamping.
- Physical AC installation and load switching.

2. System Requirements and Design Basis

2.1 Functional requirements

ID	Requirement
R1	Acquire voltage, current and power factor inputs.
R2	Calculate active power using V × I × PF.
R3	Accumulate energy in kWh and calculate cost at a configurable demonstration tariff.
R4	Detect protection faults and latch the trip state.
R5	Allow manual reset only when current and voltage are within safe limits.
R6	Provide low-PF warning with hysteresis.
R7	Maintain peak, operating-time, demand and fault-event statistics.
R8	Store selected values in EEPROM and reload them at startup.
R9	Provide LCD and serial monitoring.
R10	Support repeatable validation through a written test matrix.

2.2 Design constants

Parameter	Value
Nominal reference voltage	230 V
Simulated voltage range	180–260 V
Simulated current range	0–15 A
Simulated PF range	0.50–1.00
Overcurrent trip	I > 10 A
Overvoltage trip	V > 250 V
Undervoltage trip	V < 200 V
Low-PF warning starts	PF < 0.70
Low-PF warning clears	PF > 0.75
Electricity tariff	₹8/kWh
Maximum-demand interval	60 s demonstration window
Normal LCD screen interval	3 s
Fault/adjustment display update	200 ms
EEPROM save interval	3 s

2.3 Design philosophy

The firmware is organized as a non-blocking loop driven by millis() timing. This avoids long delays during normal operation and allows display rotation, serial reporting, EEPROM saving, demand measurement, fault display updates and LED blinking to coexist. A fault is treated as a latched state rather than a momentary condition, so returning the input to the safe region does not reconnect the simulated load automatically.

3. Hardware and System Architecture

3.1 Components

Component	Qty.	Purpose
Arduino Uno	1	Main controller; ADC inputs, control logic and EEPROM
16×2 I2C LCD	1	Live readings, statistics and fault messages
Rotary potentiometer	2	Voltage input (A0) and PF input (A2) simulation
Slide potentiometer	1	Load-current input (A1) simulation

Component	Qty.	Purpose
Relay module	1	Simulated load disconnect output
Buzzer	1	Audible protection indication
Green LED	1	Healthy/connected state indication
Red LED	1	Fault or low-PF warning indication
220 Ω resistor	2	LED current limiting
Pushbutton	1	Short press: reset; 2 s hold: input adjustment mode
Breadboard	1	Common 5 V/GND rails and interconnection

3.2 Simulated sensing method

Each potentiometer is connected between 5 V and GND. The wiper produces an ADC voltage between 0 and 5 V, which analogRead() converts to a 10-bit value from 0 to 1023. The firmware linearly maps that value to an engineering range. This is a convenient simulation method because the meter logic can be exercised without claiming that the potentiometers are actual electrical sensors.

Quantity	Analog pin	Simulated range	Firmware scaling
Voltage	A0	180–260 V	map(raw, 0, 1023, 180, 260)
Current	A1	0–15 A	map(raw, 0, 1023, 0, 1500) / 100
Power factor	A2	0.50–1.00	map(raw, 0, 1023, 50, 100) / 100

3.3 Text block diagram

SIMULATED INPUTS	ARDUINO UNO MEASUREMENT + CONTROL	OUTPUTS + HMI
Voltage pot → A0 Current slider → A1 PF pot → A2	ADC scaling Power/energy/cost Protection + logging State management	Relay control Green/red LEDs Buzzer 16×2 I2C LCD Serial monitor

3.4 Pin configuration

Pin	Type	Connection	Firmware name
A0	Analog input	Voltage potentiometer	voltagePin
A1	Analog input	Current slide potentiometer	currentPin
A2	Analog input	Power-factor potentiometer	pfPin
A4	I2C SDA	LCD SDA	Wire library
A5	I2C SCL	LCD SCL	Wire library
D2	Digital input, INPUT_PULLUP	Pushbutton to GND	resetButton
D6	Digital output	Green LED via 220 Ω	greenLED
D7	Digital output	Red LED via 220 Ω	redLED
D8	Digital output	Relay module IN	relayPin
D9	Digital output	Buzzer	buzzerPin

3.5 Software stack

- Wokwi simulator for circuit simulation and serial/LCD observation.
- Arduino C++ firmware running on the simulated Arduino Uno.
- Wire.h for I2C communication.
- LiquidCrystal_I2C for the 16×2 display.
- EEPROM.h for retained values and fault counters.

4. Firmware Design and Working Principle

4.1 Main execution sequence

1. Read the three analog inputs and scale them to voltage, current and power factor.
2. Calculate measured active power and update peak power and maximum-current records.
3. Evaluate the low power-factor warning using 0.70/0.75 hysteresis.
4. Detect overcurrent, overvoltage and undervoltage conditions.
5. Process the pushbutton for protection reset or input-adjustment mode.
6. Force effective current and effective power to zero while a protection fault is latched.
7. Integrate energy and operating time; calculate average power and cost.
8. Classify the load and accumulate the active maximum-demand window.
9. Drive the relay, LEDs and buzzer according to the current state.
10. Save retained data every 3 seconds and print the serial report every 3 seconds.
11. Update the LCD with the appropriate display priority.

4.2 State and display priority

Four major user-interface states are represented in the firmware: normal monitoring, protection fault, low power-factor warning and input-adjustment mode. Protection has highest priority. When a fault is active, the LCD shows the specific fault and the live offending value. When the value returns to the safe region, the screen changes to IN LIMIT but the fault remains latched until a safe manual reset is performed.

4.3 Input adjustment mode

A long pushbutton press of approximately 2 seconds toggles the adjustment mode when no protection fault is active. The LCD then shows live voltage, current and power factor, refreshed every 200 ms. This feature was added specifically to make simulator test setup easier; it is not essential to a physical metering product.

4.4 Low-PF warning logic

The low power-factor warning uses hysteresis. A warning begins when PF falls below 0.70 and is not cleared until PF rises above 0.75. Hysteresis prevents repeated ON/OFF transitions around one threshold. The warning does not open the relay, does not activate the buzzer, and uses a blinking red LED while the green LED remains on.

4.5 Fault-latch logic

Protection is latched. Once an overcurrent, overvoltage or undervoltage condition occurs, the relay output is forced OFF. If the measured value subsequently returns to the safe range, the relay remains OFF. A short pushbutton press is accepted only when current is at or below 10 A and voltage is between 200 V and 250 V. If an already-latched fault changes from one type to another, the relay remains OFF while the displayed and logged fault type changes accordingly.

5. Electrical Calculations and Measured Parameters

5.1 Input scaling

$$V = 180 + 80 \times rawV / 1023$$

$$I = (1500 \times rawI / 1023) / 100$$

$$PF = (50 + 50 \times rawPF / 1023) / 100$$

Because map() uses integer arithmetic, voltage changes in approximately 1 V steps, power factor in approximately 0.01 steps, and current in increments of roughly 0.015 A. These limits describe the simulated input resolution rather than the accuracy of a physical sensor.

5.2 Active power

$$P = V \times I \times PF \ [W]$$

For an AC system, this expression represents active power when V and I are RMS values and PF represents the overall power factor. In this simulation, the three quantities are independently supplied by potentiometers, so the equation demonstrates the calculation path rather than physical waveform measurement.

5.3 Energy accumulation

$$\Delta E = (P / 1000) \times (\Delta t / 3{,}600{,}000) \ \ [\text{kWh}]$$

The firmware calculates elapsed time from millis() and integrates effective power in small increments. When a protection fault is latched, effective power is forced to zero so that disconnected-load periods are not added to energy.

5.4 Cost

$$\text{Cost} = \text{Energy} \times ₹8/\text{kWh}$$

The ₹8/kWh value is a demonstration tariff selected for calculation and testing. It is not intended to represent any particular utility tariff.

5.5 Operating time and average power

$$\text{T_op accumulates only when faultState} = \text{false and sensor current} > 0$$

$$\text{P_avg} = (\text{Energy} / \text{T_op}) \times 1000 \ \ [\text{W}]$$

5.6 Peak and maximum-current records

Peak power and maximum current are updated from the measured sensor-side quantities before the effective load is zeroed by protection. Consequently, a fault-triggering sample can become the recorded maximum. These are therefore documented as highest measured values, not guaranteed highest delivered-load values.

5.7 Load classification

Effective power	Classification
$P < 500$ W	LOW LOAD
$500 \text{ W} \leq P < 1500$ W	NORMAL LOAD
$1500 \text{ W} \leq P < 2500$ W	HIGH LOAD
$P \geq 2500$ W	HEAVY LOAD

Under the current protection limits, 250 V × 10 A × PF 1.00 = 2500 W is the maximum non-tripped operating point. Therefore the HEAVY LOAD band is technically reachable only at this exact boundary condition and is effectively a corner case.

5.8 Maximum demand

$$\text{P_demand} = \text{E_window} / (60/3600) \times 1000 \ \ [\text{W}]$$

The firmware uses a fixed 60-second window for practical demonstration in Wokwi. At the end of each completed window, the average power in that window is compared with the stored maximum demand. Utility demand meters often use longer demand intervals, so this implementation is intentionally a demonstration model.

5.9 Worked example

For a steady operating point of 230 V, 5.00 A and PF 0.90:

Quantity	Calculation	Result
Active power	230 × 5.00 × 0.90	1035 W
Energy after 10 min	1.035 kW × 10/60 h	0.1725 kWh
Cost after 10 min	0.1725 × 8	₹1.38
Energy after 1 h	1.035 kW × 1 h	1.035 kWh
Cost after 1 h	1.035 × 8	₹8.28
Average power after 1 h	1.035 kWh ÷ 1 h × 1000	1035 W
One completed 60 s demand window	0.01725 kWh ÷ (1/60 h) × 1000	1035 W

6. Protection, Monitoring and Data Retention

6.1 Protection thresholds

Protection	Trip condition	Priority
Overcurrent	I > 10 A	1
Overvoltage	V > 250 V	2
Undervoltage	V < 200 V	3

The comparisons are strict. Exactly 10 A, 250 V and 200 V remain within the healthy boundary. If more than one condition is present simultaneously, overcurrent has the highest priority, followed by overvoltage and undervoltage.

6.2 Protection outputs

State	Relay	Green LED	Red LED	Buzzer
Normal	ON	ON	OFF	OFF
Low-PF warning	ON	ON	Blinking	OFF
Protection fault	OFF	OFF	ON	ON

6.3 Dynamic fault display

The protection screen updates every 200 ms without continuously clearing the LCD. This prevents visible stutter. The top line identifies the fault type; the lower line reports the live value and either the violated limit or IN LIMIT. Examples are OVERCURRENT with a current above/below 10 A, OVERVOLTAGE with a voltage above/below 250 V, and UNDERVOLTAGE with a voltage below/above 200 V.

6.4 EEPROM data storage

Stored value	Address
Energy (kWh)	0
Peak power (W)	10
Maximum current (A)	20
Load operating time (h)	30
Maximum demand (W)	40
Overcurrent events	50
Overvoltage events	60
Undervoltage events	70
EEPROM signature	80

The firmware uses EEPROM.get() to load retained values at startup and EEPROM.put() to save them. A format signature (0xA217) helps distinguish the current operating-time data layout from an older layout. Invalid floating-point values and implausibly large event counters are cleared to zero during initialization.

6.5 LCD information structure

Screen	Information
0	Voltage, current, PF and power
1	Power, PF and voltage
2	Energy
3	Cost
4	Peak power
5	Maximum current
6	Average power
7	Load status
8	Maximum demand
9	Protection-event counts

Normal screens rotate approximately every 3 seconds. Display priority is protection fault → input adjustment mode → low-PF warning → normal monitoring rotation.

7. Testing and Validation

7.1 Test methodology

The final firmware was evaluated in the Wokwi simulator from a known healthy starting condition. Potentiometers were adjusted to produce controlled voltage, current and PF combinations. Results were observed through the LCD, serial monitor, relay state, LEDs and buzzer. Protection tests were validated as full sequences—trip, return toward the safe range, safe/unsafe reset attempt and final reset—rather than as single trigger events.

7.2 Final 24-case validation matrix

No.	Test	Action	Expected observation	Result
1	Normal operation	~230 V, 5 A, PF 0.90–1.00	Correct readings; relay ON; green ON	PASS
2	Enter adjustment mode	Hold button ~2 s	Live V/I/PF; normal rotation stops	PASS
3	Exit adjustment mode	Hold button ~2 s again	Normal screens resume	PASS
4	Overcurrent trip	Raise current above 10 A	Fault latch; relay OFF; red ON; buzzer ON	PASS
5	Overcurrent recovery	Lower current below 10 A	IN LIMIT; relay remains OFF	PASS
6	Overcurrent reset	Short press when safe	Fault clears; relay ON	PASS
7	Overvoltage trip	Raise voltage above 250 V	OV fault displayed; relay OFF	PASS
8	Overvoltage recovery	Return to 200–250 V	IN LIMIT; reset permitted	PASS
9	Overvoltage reset	Short press when safe	Normal operation restored	PASS
10	Undervoltage trip	Lower voltage below 200 V	UV fault displayed; relay OFF	PASS
11	Undervoltage recovery	Return to 200–250 V	IN LIMIT; reset permitted	PASS
12	UV → OV while latched	Move from <200 V to >250 V	Displayed fault changes to OV; relay stays OFF	PASS
13	OV → UV while latched	Move from >250 V to <200 V	Displayed fault changes to UV; relay stays OFF	PASS
14	Reset blocked	Press while values remain unsafe	Relay stays OFF	PASS
15	Low PF warning	PF <0.70 with V/I safe	Warning; red LED blinks; relay ON; buzzer OFF	PASS
16	PF recovery	Raise PF above 0.75	Warning clears	PASS
17	Energy calculation	Fixed load for known time	Energy increases according to P × t	PASS
18	Cost calculation	Continue energy accumulation	Cost = energy × ₹8	PASS
19	Peak values	Apply increasing load	Highest measured power/current retained	PASS
20	Load classification	Traverse low/normal/high/heavy points	Correct class reported	PASS
21	Maximum demand	Steady load across 60 s	Completed-window average stored	PASS
22	EEPROM retention	Save values, restart simulation	Retained firmware values reload as designed	PASS
23	Fault logging	Trigger each fault type	Matching event counter increments	PASS
24	Fault latch	Trigger fault, change input without reset	Relay remains OFF until safe reset	PASS

7.3 Reference calculation checks

Inputs	Calculated power	Expected load status
230 V, 2.00 A, PF 1.00	460 W	LOW LOAD
230 V, 5.00 A, PF 0.90	1035 W	NORMAL LOAD
230 V, 8.00 A, PF 0.90	1656 W	HIGH LOAD
250 V, 10.00 A, PF 1.00	2500 W	HEAVY LOAD

7.4 Result discussion

The protection sequence tests are the strongest evidence that the firmware behaves as a control system rather than merely as a calculator. The relay is disconnected on fault, the fault stays latched while the measured condition changes, and the reset is blocked until the inputs are safe. The low-PF test demonstrates a separate warning state with hysteresis instead of unnecessary load disconnection. Calculation tests agree with the implemented equations, while event counters and retained statistics provide evidence that the data-management layer operates independently from the live measurement layer.

Because all sensor values are ideal simulator inputs, the validation confirms firmware logic, timing and state transitions—not real electrical measurement accuracy. A physical build would require sensor calibration, isolation and safety testing before any mains connection.

8. Limitations and Design Review

8.1 Simulation limitations

- Potentiometers provide ideal, noise-free values. Sensor noise, offset, gain error and calibration are not tested.
- Voltage and current waveforms are not sampled. True RMS values and phase displacement are not derived from waveform data.
- Power factor is supplied independently rather than calculated from voltage/current phase information.
- The tariff is a flat demonstration rate and events have no RTC timestamps.
- The pushbutton is not software-debounced; a real switch would normally require debouncing.

8.2 Known design issues before physical deployment

1. EEPROM write frequency. The firmware writes cumulative values every 3 seconds. For a physical ATmega328P implementation, repeated writes to the same EEPROM locations would eventually approach the device endurance rating. A write-on-change strategy, larger storage ring or wear-leveling technique would be preferable.

2. Floating-point energy accumulation. Energy is stored as a 32-bit float and updated by small loop-level increments. Over a very long operating period, the resolution of the float becomes a concern. Integer accumulation in Wh or mWh, or a wider numeric representation, would provide better deterministic resolution.

3. HEAVY LOAD threshold reachability. The current protection ceiling makes 2500 W the highest non-tripped point, and that value is also the HEAVY threshold. Consequently the HEAVY class is only reachable at the exact 250 V, 10 A and PF 1.00 corner. This is acceptable for demonstrating the logic but would normally be redesigned for better classification separation.

4. Peak metrics include trip-causing samples. The highest measured sensor-side value is retained even when that same sample triggers protection. This is intentionally documented as a measured peak, not a delivered-load peak.

5. Maximum-demand interval. The demonstration uses a 60-second window. A production design could use a configurable interval or a utility-aligned sliding/rolling method.

6. Relay polarity. The simulation assumes a HIGH control signal corresponds to an energized relay. A physical relay module may be active-low and must be verified before wiring the control logic to an actual load.

9. Future Development

Area	Future improvement
Physical sensing	Replace potentiometers with an isolated voltage-sensing stage and a properly rated current transformer or Hall-effect sensor.
Electrical measurement	Add RMS calculation, waveform sampling and phase-based PF determination.
Calibration	Calibrate against a reference meter and quantify measurement error.
Data storage	Use wear-aware EEPROM/FRAM strategy and integer or higher-resolution energy accumulation.
Communication	Move to an ESP32-class controller for Wi-Fi/BLE monitoring and remote dashboards.
Time and events	Add RTC/network time so fault events can be timestamped.
Tariff	Support configurable slab and time-of-day tariffs.
Demand	Use a longer or configurable demand period suited to the target application.
Safety	Implement physical isolation, fusing/MCB, enclosure, clearances and a properly rated switching device before any mains testing.

10. Conclusion

The Smart Energy Meter project successfully demonstrates a multi-function embedded energy-monitoring and protection system in a safe simulation environment. The final firmware measures simulated voltage, current and power factor; calculates active power, energy and cost; records operating time, average power, peaks and maximum demand; classifies load conditions; and presents the results through a rotating LCD and serial monitor.

The protection subsystem is the key engineering feature. Overcurrent, overvoltage and undervoltage conditions cause a latched trip, while a dedicated low-PF warning uses hysteresis without unnecessarily disconnecting the load. EEPROM-backed statistics and fault counters add a data-retention layer, and the 24-case validation matrix provides a repeatable record of the final behaviour.

The project is intentionally positioned as a simulation-based prototype rather than a commercial meter. Its strongest value for further development is the firmware architecture: sensor abstraction, timed processing, state latching, protection priority, HMI behaviour, persistence and systematic testing are all established. The next engineering step would be to replace the simulated inputs with isolated, calibrated sensing hardware and then validate the complete system against real electrical measurements.

References

No.	Source	Reference	Use in project
1	Microchip Technology Inc.	ATmega328/P Datasheet.	Analog-to-digital converter, EEPROM and microcontroller specifications.
2	Arduino	Arduino Language Reference and Libraries.	analogRead(), map(), millis(), Wire and EEPROM.
3	Wokwi	Wokwi Documentation.	Arduino Uno and simulation components used by the project.
4	MIT OpenCourseWare	Technical report and project-report guidance.	Used as a structural reference for abstract, introduction, methodology/results, conclusions and appendices.
5	IEC	IEC 62053-21, Electricity metering equipment – Particular requirements – Static meters for active energy (classes 1 and 2).	Referenced only for general commercial-metering context; this project makes no compliance claim.

Official online sources: Arduino Reference | Wokwi Documentation | Microchip ATmega328P

Appendix A. Project Resources and Reproducibility

The final tested Arduino C++ source code and Wokwi circuit configuration are maintained separately from this report. The source files used for this documentation were checked for consistency with the pin configuration, protection thresholds, EEPROM addresses, display behaviour and measurement scaling described above.

The Wokwi circuit configuration contains the Arduino Uno, LCD1602 I2C display, two rotary potentiometers, one slide potentiometer, relay module, buzzer, green/red LEDs, two 220 Ω resistors, pushbutton and breadboard connections. No AC mains source or physical load is present in the simulation.

Project Links

Wokwi Simulation: https://wokwi.com/projects/470533202965811201
GitHub Repository: https://github.com/SohanHR/smart-energy-meter

Appendix B. Engineering Notes for a Physical Prototype

- Do not connect the simulated circuit directly to mains voltage.
- Use an isolated voltage-sensing stage and properly rated current sensor.
- Use an appropriately rated relay/contactor, fuse or MCB, enclosure and terminal system.
- Provide adequate creepage, clearance, earthing and overvoltage protection.
- Validate protection thresholds against calibrated instruments before connecting any real load.